

Triple Peaks

Die Cordillera Oriental ist ein Gebirgszug in den Anden, die sich über Bolivien erstreckt. Sie besteht aus einer Reihe von N Berggipfeln, die von 0 bis $N - 1$ nummeriert sind. Die **Höhe** des Gipfels i ($0 \leq i < N$) ist $H[i]$, eine ganze Zahl zwischen 1 und $N - 1$, einschließlich.

Für zwei beliebige Gipfel i und j , wobei $0 \leq i < j < N$, ist die **Distanz** zwischen ihnen definiert als $d(i, j) = j - i$.

Nach alten Inka-Legenden ist eine Dreiergruppe von Gipfeln **mythisch**, wenn sie die folgende besondere Eigenschaft aufweist: die Höhen der drei Gipfel **matchen** mit ihren paarweisen Distanzen, wenn sie, **unbeachtet der Reihenfolge**, gleich sind.

Formal ist eine Dreiergruppe mit den Indizes (i, j, k) mythisch, wenn

- $0 \leq i < j < k < N$, und
- die Höhen $(H[i], H[j], H[k])$ mit den paarweisen Distanzen $(d(i, j), d(i, k), d(j, k))$ matchen. Beispielsweise betragen für die Indizes 0, 1, 2 die paarweisen Distanzen $(1, 2, 1)$, sodass $(H[0], H[1], H[2]) = (1, 1, 2)$, $(H[0], H[1], H[2]) = (1, 2, 1)$ und $(H[0], H[1], H[2]) = (2, 1, 1)$ alle mit den Distanzen matchen, die Höhen $(H[0], H[1], H[2]) = (1, 2, 2)$ jedoch nicht.

Diese Aufgabe besteht aus zwei Teilen, wobei jeder Subtask entweder mit **Teil I** oder **Teil II** verbunden ist. Du kannst die Subtasks in beliebiger Reihenfolge lösen. Insbesondere ist es **nicht** erforderlich, dass du Teil I vollständig löst, bevor du Teil II in Angriff nimmst.

Teil I

Anhand einer Beschreibung des Gebirges sollst du die Anzahl der mythischen Dreiergruppen zählen.

Implementierungsdetails

Du sollst die folgende Funktion implementieren.

```
long long count_triples(std::vector<int> H)
```

- H : Array der Länge N , das die Höhen der Gipfel darstellt.
- Diese Funktion wird für jeden Testfall genau einmal aufgerufen.

Die Funktion soll eine ganze Zahl T zurückgeben, die die Anzahl der mythischen Dreiergruppen im Gebirge angibt.

Einschränkungen

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$ für jedes i , so dass $0 \leq i < N$.

Subtasks

Teil I ist insgesamt 70 Punkte wert.

Subtask	Punkte	Zusätzliche Beschränkungen
1	8	$N \leq 100$
2	6	$H[i] \leq 10$ für alle i ($0 \leq i < N$).
3	10	$N \leq 2000$
4	11	Die Höhen nehmen nicht ab. Also gilt $H[i - 1] \leq H[i]$ für alle i ($1 \leq i < N$).
5	16	$N \leq 50\,000$
6	19	Keine zusätzlichen Beschränkungen.

Beispiel

Betrachte den folgenden Funktionsaufruf.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Es gibt 3 mythische Dreiergruppen in der Bergkette:

- Bei $(i, j, k) = (1, 3, 4)$ matchen die Höhen $(1, 3, 2)$ mit den paarweisen Distanzen $(2, 3, 1)$.
- Für $(i, j, k) = (2, 3, 6)$ matchen die Höhen $(4, 3, 1)$ mit den paarweisen Distanzen $(1, 4, 3)$.
- Für $(i, j, k) = (3, 4, 6)$ matchen die Höhen $(3, 2, 1)$ mit den paarweisen Distanzen $(1, 3, 2)$.

Folglich sollte die Funktion 3 zurückgeben.

Beachte, dass die Indizes $(0, 2, 4)$ kein mythisches Tripel bilden, da die Höhen $(4, 4, 2)$ nicht mit den paarweisen Distanzen $(2, 4, 2)$ matchen.

Teil II

Deine Aufgabe ist es, Gebirgszüge mit vielen mythischen Tripeln zu konstruieren. Dieser Teil besteht aus 6 **Output-Only** Subtasks mit **Partial Scoring**.

In jedem Subtask sind zwei positive ganze Zahlen M und K gegeben, und du sollst ein Gebirge mit **maximal** M Gipfeln konstruieren. Wenn deine Lösung **mindestens** K mythische Dreiergruppen enthält, erhältst du die volle Punktzahl für diesen Subtask. Andernfalls ist deine Punktzahl proportional zur Anzahl der mythischen Dreiergruppen in deiner Lösung.

Beachte, dass deine Lösung aus einer gültigen Gebirgskette bestehen muss. Nehmen wir an, deine Lösung hat N Gipfel (mit $3 \leq N \leq M$). Dann muss die Höhe des Gipfels i ($0 \leq i < N$), bezeichnet mit $H[i]$, eine ganze Zahl zwischen 1 und $N - 1$, einschließlich, sein.

Implementierungsdetails

Es gibt zwei Methoden, um deine Lösung einzureichen, und du kannst für jeden Subtask eine der beiden Methoden verwenden:

- **Ausgabedatei**
- **Aufruf der Funktion**

Um deine Lösung über eine **Ausgabedatei** einzureichen, erstelle und übermittle eine Textdatei im folgenden Format:

```
N
H[0] H[1] ... H[N-1]
```

Um deine Lösung über einen **Funktionsaufruf** zu übermitteln, implementiere folgende Funktion.

```
std::vector<int> construct_range(int M, int K)
```

- M : die maximale Anzahl an Gipfeln.
- K : die gewünschte Anzahl der mythischen Tripel.
- Diese Funktion wird für jeden Subtask genau einmal aufgerufen.

Die Funktion sollte ein Array H der Länge N zurückgeben, das die Höhen der Gipfel darstellt.

Subtasks und Punkte

Teil II ist insgesamt 30 Punkte wert. Für jeder Subtask sind die Werte von M und K festgelegt und in der folgenden Tabelle angegeben:

Subtask	Punkte	M	K
7	5	20	30
8	5	500	2000
9	5	5000	50 000
10	5	30 000	700 000
11	5	100 000	2 000 000
12	5	200 000	12 000 000

Wenn deine Lösung bei einem Subtask kein gültiges Gebirge bildet, wird sie mit der Punktzahl 0 bewertet (im CMS als `Output isn't correct` angegeben).

Andernfalls sei T die Anzahl der mythischen Dreiergruppen in deiner Lösung. Dann lautet deine Punktzahl für den Subtask:

$$5 \cdot \min \left(1, \frac{T}{K} \right).$$

Beispiel-Grader

Für die Teile I und II wird derselbe Beispiel-Grader verwendet, wobei durch die erste Zeile der Eingabe zwischen den beiden Teilen unterschieden wird.

Eingabeformat für Teil I:

```
1
N
H[0] H[1] ... H[N-1]
```

Ausgabeformat für Teil I:

```
T
```

Eingabeformat für Teil II:

```
2
M K
```

Ausgabeformat für Teil II:

N

H[0] H[1] ... H[N-1]

Beachte, dass die Ausgabe des Beispiel-Graders dem erforderlichen Format für die Ausgabedatei in Teil II entspricht.